SBC Summer School 2026

DFS Avançada

Bernardo Amorim

30 de janeiro de 2026

Plano de aula

- DFS
- Árvore da DFS
- Pontes
- Pontos de Articulação
- Árvore da DFS para Grafos Direcionados
- DAG de Componentes Fortemente Conexos
- Algoritmo de Tarjan

Algoritmo DFS

```
vector<bool> vis(N);
vector<vector<int>> g(N);

void dfs(int v) {
    vis[v] = true;

    for (int u : g[v]) {
        if (!vis[u]) {
            dfs(u);
        }
    }
}
```

Complexidade: $\mathcal{O}(N + M)$

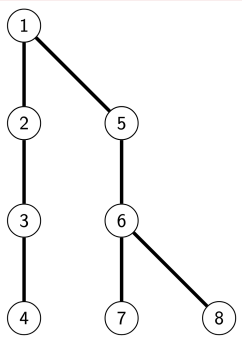
Definições

Definição

Grafo conexo: um grafo em que existe um caminho entre qualquer par de vértices.

Definição

Árvore: um grafo conexo sem ciclos.



Definições

Definição

Árvore enraizada: árvore percorrida partindo de uma raiz (primeiro vértice a ser visitado).

Nota

Usualmente desenhamos uma árvore com a raiz em cima.

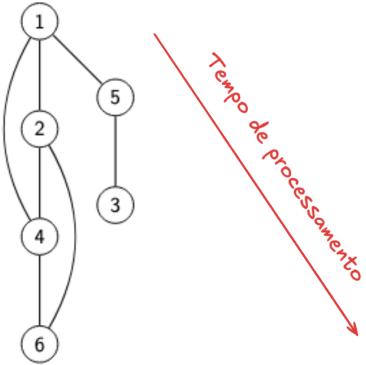
Definição

Ancestral: u é ancestral de v se u é visitado antes de v e o caminho da raiz até v passa por u.

Definição

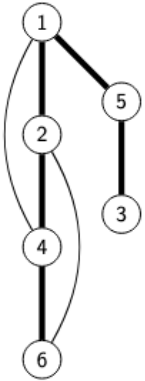
Descendente: v é descendente de u se u é ancestral de v.

Árvore da DFS



- Representar segundo o tempo de processamento é muito conveniente.

Tipos de arestas



- Span edges - arestas que foram usadas para visitar um novo vértice na DFS. Induzem uma árvore geradora do grafo (*spanning tree*), a árvore da DFS.
- Back edges - arestas que conectam um vértice a um ancestral (que não é seu pai) na árvore da DFS.

Propriedades

Lema

Se uma aresta não é uma span edge, ela é uma back edge.

Demonstração.

Considera uma aresta uv e suponha, sem perda de generalidade que u foi visitado antes de v.

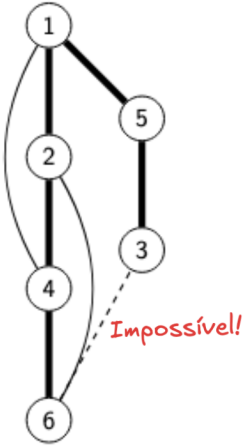
- Se v foi visitado usando uv, uv é uma span edge.
- Caso contrário, quando uv foi considerada, v já tinha sido visitado. Logo, v foi visitado a partir de vizinhos de u, logo v é descendente de u na árvore da DFS. Assim uv é uma back edge.

$\square$

Eureka!

Isso é o que torna a "árvore da DFS tão útil!"

O grafo tem uma estrutura simples em que toda aresta ou faz parte da árvore geradora, ou liga um vértice com seu ancestral.



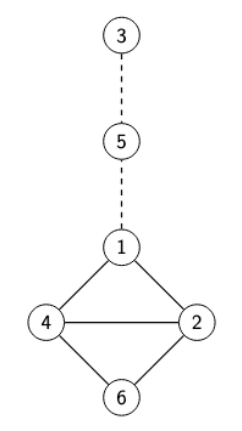
Correspondencia no algoritmo DFS

```
1  void dfs(int v) {
2      vis[v] = true;
3
4      for (int u : g[v]) {
5          if (!vis[u]) { // uv é span edge
6              dfs(u);
7          } // else uv é back edge
8      }
9  }
```

Pontes

Definição (Pontes)

Uma ponte é uma aresta cuja remoção aumenta o número de componentes conexos do grafo.



Propriedades

Lema

Uma back edge nunca é ponte.

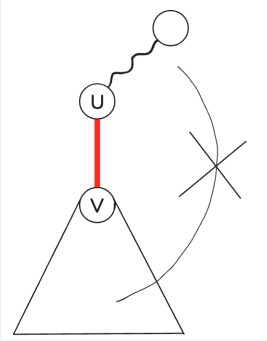
Demonstração.

Se uv é uma back edge, então existe caminho $u - v$ através de span edges (já que são geradoras do grafo), logo uv não é ponte. $\square$

Propriedades

Lema

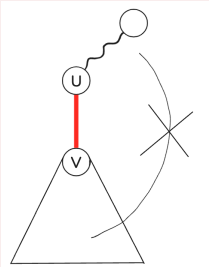
Uma span edge uv é uma ponte se e somente se não existe aresta que conecta um descendente de uv com um ancestral de uv.



Propriedades

Demonstração.

Suponha, sem perda de generalidade, que visitamos u antes de v.

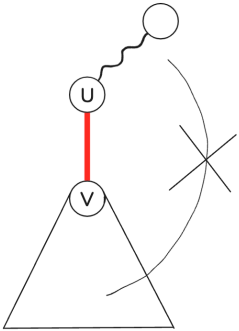


($\Rightarrow$) Se uv é ponte, então a subárvore de v na árvore da DFS está desconectada do componente de u, logo não há aresta conectando descendente de uv com um ancestral de uv.
($\Leftarrow$) Se não existe aresta conectando um descendente de uv com ancestral de uv, então a subárvore de v está desconectada do restante da árvore, logo uv é ponte. $\square$

Ideia do algoritmo

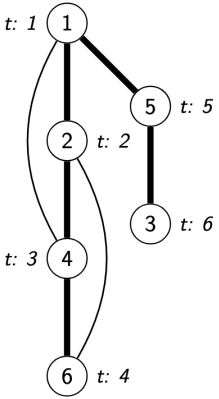
- Toda aresta é uma span edge ou uma back edge.
- Uma back edge nunca é ponte (todas as pontes são span edges).
- *uv* é ponte se e somente se não existe aresta que "atravessa" *uv* na árvore da DFS.

Agora fica claro resolver o problema, basta fazermos a árvore da DFS e, para cada span edge, checamos se existe alguma aresta que atravessa ela, se não, temos uma ponte.



Contador

Um truque útil é adicionar um contador global de tempo na DFS para saber a ordem de visitação dos vértices.



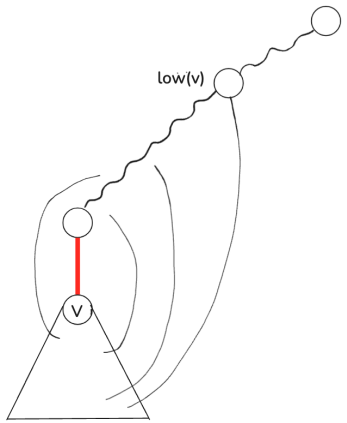
Implementação contador

```
vector<int> in(N);

void dfs(int v, int& t) {
    vis[v] = true;
    in[v] = t++;

    for (int u : g[v]) {
        if (!vis[u]) { // uv é span edge
            dfs(u, t);
        } // else uv é back edge
    }
}
```

Low definição

Definição

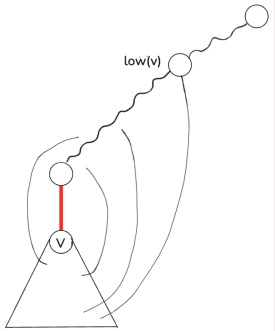
low(v) *é o tempo de chegada (in) do vértice mais alto alcançável por uma back edge partindo da subárvore de* v *na árvore da DFS ou* in(v) *caso tal aresta não exista.*



Low fórmula

Lema

$$low(v) = \min \begin{cases} low(u), & u \in \textit{filhos de } v \\ in(u), & \forall uv | uv \textit{ é back edge} \\ in(v) & \end{cases}$$



Low implementação

```
vector<int> low(N);

int dfs(int v, int& t, int p = -1) { // retorna low(v)
    vis[v] = true;
    in[v] = low[v] = t++;

    for (int u : g[v]) {
        if (!vis[u]) { // uv é span edge
            low[v] = min(low[v], dfs(u, t, v));
        } else { // uv é back edge*, a menos que u seja o pai de v
            if (u != p) { // checando se é filho de v
                low[v] = min(low[v], in[u]);
            }
        }
    }
    return low[v];
}
```

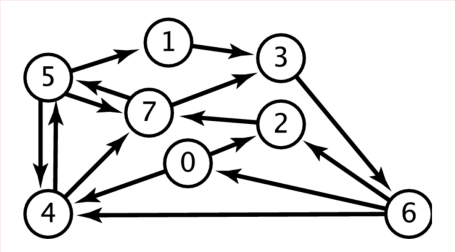
Algoritmo pontes

```
int dfs(int v, int& t, int p = -1) { // retorna low(v)
    vis[v] = true;
    in[v] = low[v] = t++;

    for (int u : g[v]) {
        if (!vis[u]) { // uv é span edge
            low[v] = min(low[v], dfs(u, t, v));
        } else if (u != p) { // uv é back edge
            low[v] = min(low[v], in[u]);
        }
    }

    // para toda span edge, checamos no vértice de baixo se ela é ponte
    if (p != -1 and low[v] == in[v]) {
        // vp é ponte
    }
}
```

Grafo fortemente conexo

Definição

Um grafo direcionado é dito ser fortemente conexo se existe um caminho de todo vértice para todo outro vértice.



Definição

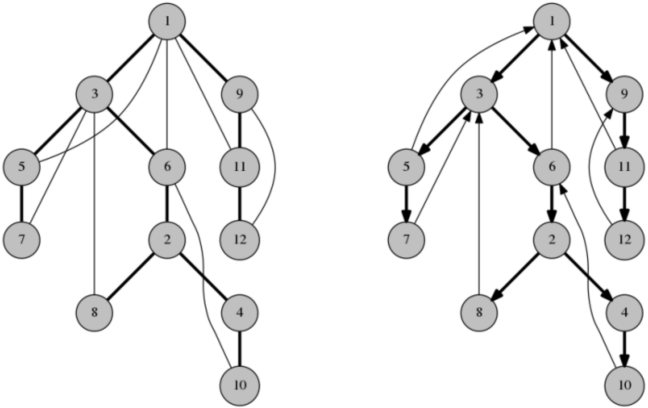
Um componente fortemente conexo é um subgrafo fortemente conexo maximal - todos os vértices visitam todos e, adicionar qualquer outro vértice ao componente quebraria essa propriedade.

Problema Bertown Roads

Direcione as arestas do grafo simples para torná-lo fortemente conexo ou declare impossível.

Teorema

Um grafo pode ser orientado para ser fortemente conexo se e somente se ele não tem pontes.



Solução Bertown Roads

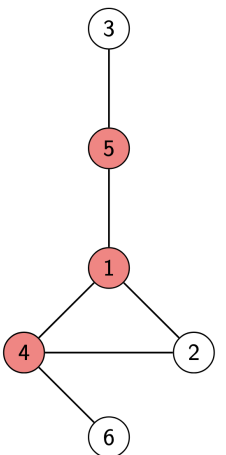
Demonstração.

($\Rightarrow$) Assuma sem perda de generalidade que orientamos a ponte uv no sentido $u \to v$. v não consegue alcançar u pois não tem outro caminho $u - v$ uma vez que uv é ponte.
($\Leftarrow$) Oriente todas as span edges "para baixo" e todas as back edges "para cima". Como não há pontes todo vértice consegue subir pelas back edges até a raiz e aí chegar a qualquer outro vértice pelas span edges, logo o grafo é fortemente conexo. $\square$

Ponto de articulação

Definição

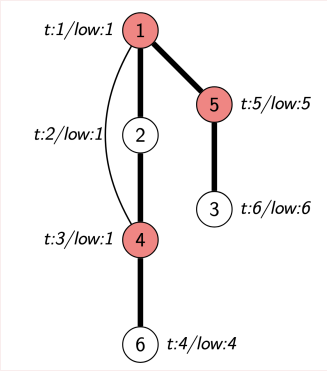
Um ponto de articulação é um vértice cuja remoção aumenta o número de componentes conexos do grafo.



Lema dos pontos de articulação

Lema

Se v é raiz da árvore da DFS, v é ponto de articulação se e somente se tem mais de um filho. Caso contrário, v é ponto de articulação se e somente se tem filho u tal que $low(u) \geq in(v)$.

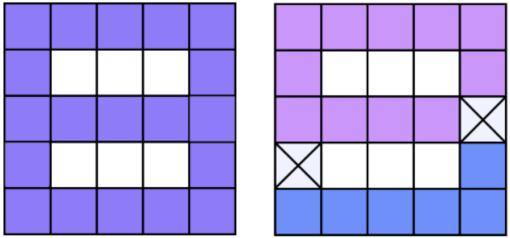


Prova pontos de articulação

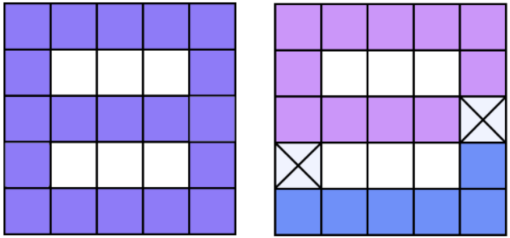
Demonstração.

Se v é raiz, o Lema segue do fato de que toda aresta é back edge ou span edge - os filhos da raiz não se conectam por outra via senão a raíz.
Quando v não é raiz, ter um filho u tal que $low(u) \geq in(v)$ implica que não existe caminho $u \rightarrow v$ sem passar pela aresta uv (caso contrário $low(u) < in(v)$), logo a remoção de v aumenta o número de componentes. $\square$

Solução Cutting Figure



Solução Cutting Figure



A resposta é no máximo 2 (dá pra isolar uma peça no cantinho).
Checar se a resposta é 1 usando o algoritmo para checar pontos de articulação.

Problema Fairy

Descubra quais arestas tornam o grafo bipartido ao serem removidas do grafo.

Nota

Um grafo é bipartido se e somente se não possui ciclos ímpares.

Ideias:

- Após montar a árvore da DFS, podemos colorir os vértices alternadamente.
- Teremos arestas "contraditórias" (ligam vértices de mesma cor).

Solução Fairy

Lema

Uma back edge é resposta se e somente si ela é a única aresta contraditória.

Demonstração.

($\Rightarrow$) Se houvesse outra aresta contraditória, não haveria solução pois haveria ciclo ímpar passando pelas span edges.
($\Leftarrow$) Evidente. $\square$

Solução Fairy

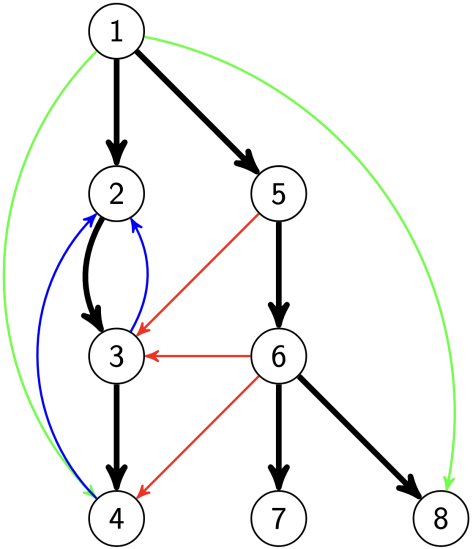
Lema

Uma span edge é resposta se e somente se todas as arestas que a cruzam são exatamente as arestas contraditórias.

Demonstração.

($\Rightarrow$) Se houvesse aresta contraditória que não cruza essa span edge, não haveria solução pois continuamos tendo um ciclo ímpar passando por outras span edges.
($\Leftarrow$) Podemos inverter a subárvore abaixo da aresta removida e passa a ser uma solução. $\square$

Árvore da DFS direcionada



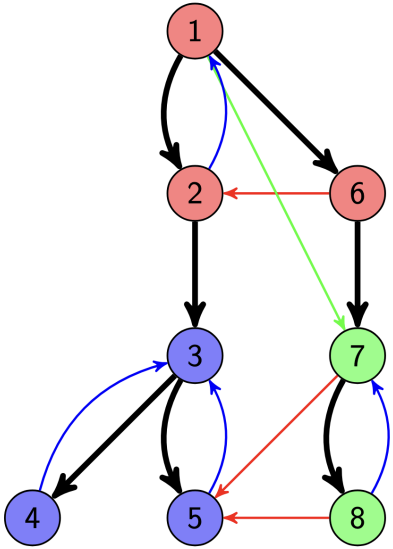
Tipos de arestas direcionadas

- Span edges - arestas que foram usadas para visitar um novo vértice na DFS. Induzem uma árvore geradora do grafo (*spanning tree*), a árvore da DFS.
- Back edges (azuis) - arestas que conectam um vértice a um ancestral na árvore da DFS.
- Forward edges (verdes) - arestas que conectam um vértice a um descendente na árvore da DFS.
- Cross edges (vermelhas) - arestas que conectam um vértice com um vértice visitado antes na DFS e que não é seu ancestral.

Nota

Todas as arestas caem em alguma dessas categorias.

Componentes Fortemente Conexos



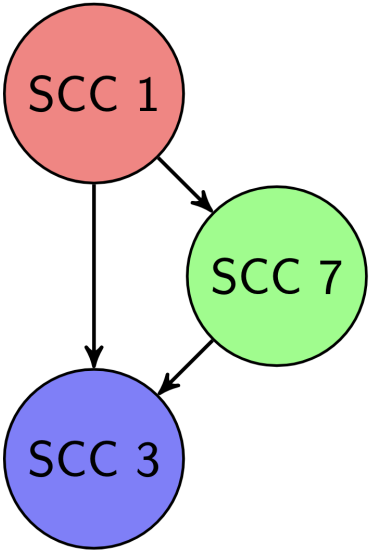
Lema

Os vértices de um componente fortemente conexo (SCC) induzem um subgrafo conexo da árvore da DFS.

Demonstração.

Considere o vértice mais alto do SCC (r, o representante) e um outro vértice qualquer do SCC, v. Todos os vértices no caminho de v até r na árvore da DFS fazem parte do SCC, logo ele é conexo. □

DAG de SCC



- Ao comprimir os componentes fortemente conexos (transformá-los em vértices usando um representante), ficamos com um DAG - um grafo direcionado sem ciclos (se houvesse um ciclo, todos os SCCs nesse ciclo seriam parte do mesmo componente.
- DAGs tem a ótima propriedade de poderem ser ordenados.

Problema Checkposts

Queremos adicionar alguns postos policiais na cidade de forma tal que, para todo vértice v, exista algum vértice u com posto policial tal que seja possível ir de v para u e voltar. Colocar um posto policial tem custo diferente para cada vértice.

Problema Checkposts

Queremos adicionar alguns postos policiais na cidade de forma tal que, para todo vértice v, exista algum vértice u com posto policial tal que seja possível ir de v para u e voltar. Colocar um posto policial tem custo diferente para cada vértice.
Queremos colocar um posto policial no vértice mais barato de cada SCC.

Lema dos vértices não rotulados

Lema

Todos os vértices na subárvore de v cujo representante de seu SCC não é descendente de v estão no SCC de v.

Demonstração.

O representante de qualquer vértice u é um ancestral seu (ou o próprio u). Como esse ancestral não é descendente de v, ele é v ou ancestral de v. Logo v faz parte do componente uma vez que temos caminhos $r \to v \to u \to r$. $\square$

Nota

Ver subárvore do vértice 1 no slide anterior.

Lema da raiz

Lema

Um vértice v é a raiz de seu SCC se e somente se $low(v) = in(v)$.

Demonstração.

($\Rightarrow$) Se $\text{low}(v) \neq \text{in}(v)$ então $\text{low}(v) < \text{in}(v)$, o que significa que existe caminho v para um ancestral seu passando por sua subárvore. Isso implica que seu ancestral estaria no SCC de v, logo v não seria raiz.
($\Leftarrow$) A igualdade implica que nenhum vértice alcança um ancestral de v, logo v é o vértice mais alto do SCC. $\square$

Algoritmo de Tarjan

```
vector<int> comp(N), vis(N); // 0 não visitado, 1 em rotulação, 2 rotulado
stack<int> st;
int dfs(int v, int& t, int p = -1) { // retorna low(v)
    vis[v] = 1;
    in[v] = low[v] = t++;
    st.push(v);
    for (int u : g[v]) {
        if (!vis[u]) { // uv é span edge
            low[v] = min(low[v], dfs(u, t, v));
        } else if (vis[u] == 1) { // uv é back, cross ou
        // forward edge para não rotulado. Apenas essas arestas são úteis pois
        // se o vértice já foi rotulado, ele não consegue alcançar v.
            low[v] = min(low[v], in[u]);
        }
    }
    // Compressão de SCC
    return low[v];
}
```

Algoritmo de Tarjan

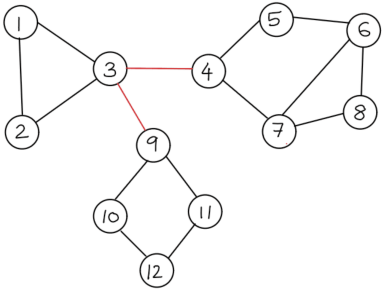
```cpp
vector<int> comp(N), vis(N); // 0 não visitado, 1 em rotulação, 2 rotulado
stack<int> st;
int dfs(int v, int& t, int p = -1) { // retorna low(v)
    // Iniciação e DFS
    if (low[v] == in[v]) { // v é o representante do SCC
        while (true) {
            int u = st.top(); st.pop();
            vis[u] = 2, comp[u] = v;
            if (u == v) break;
        }
    }
    return low[v];
}
```

Notas Algoritmo de Tarjan

- Complexidade $\mathcal{O}(N + M)$
- Diferentemente da DFS tree para grafos não direcionados, para grafos direcionados pode ser preciso chamar a DFS a partir de todos os vértices pois, a depender do grafo, pode ser que a escolha de raiz não atinja todos os vértices.
- O mesmo algoritmo pode ser usado para comprimir ciclos em grafos não direcionados, basta não voltar para o pai.

Bridge Tree

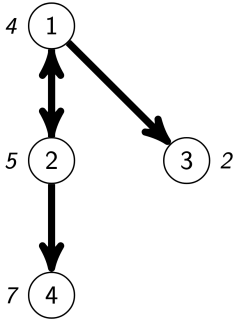
- Usando a mesma lógica de compressão de SCCs para o caso não direcionado, formamos a "bridge tree" do grafo.



Problema Coin Collector

Temos um grafo direcionado em que cada vértice tem uma quantidade de moedas. Você pode andar pelas arestas do grafo e coletar as moedas que tem lá (não tem problema repetir aresta/vértice, mas depois de ter pego as moedas elas não voltam).

Qual é o máximo número de moedas que você pode coletar dado que você pode escolher o lugar de início e fim?



Problema Coin Collector

- Conseguimos coletar todas as moedas em um SCC.
- Montamos o DAG de SCC - o valor de cada SCC é a soma dos vértices contidos nele.
- Fazemos uma DP nessa DAG para calcular o caminho de maior valor.

Formulário feedback

https://forms.gle/jzqS5eHyp6QZAj2p7

Referências

- DFS Tree and its Applications
- SCC CP Algorithms
- Biblioteca UFMG
- Cutpoints CP Algorithms
- Strong Orientation CP Algorithms
- Bridge Tree